

Machine Learning Report

ALESSANDRO MASSIMO MERMALI

March 21st 2026

1 Exercise 1

In the file `regression1.csv`, a data set is collected containing the values sampled from an unknown function. Identify the best approach to use to build a regression model for this dataset, and implement a Python script `exercise1.py` that can be used to generate a csv file (named `result1.csv`) containing the values computed by the function in the interval of the dataset with a step of 0.25.

The initial step involved plotting the dataset. As seen in Figure 1, the

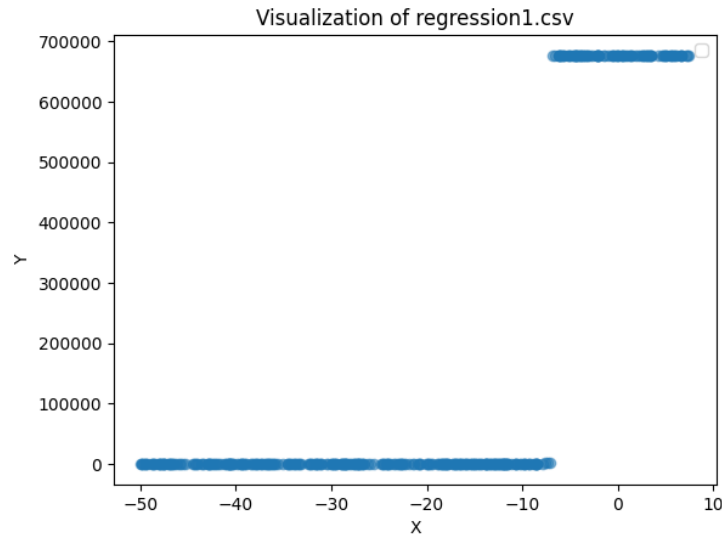


Figure 1: Visualization of regression1.csv

function appears as a step function. This suggests the use of **Decision Trees** might be crucial into finding the right prediction model. It's also critical to notice that the data is widely uneven, ranging from values around 50 to values around 60000. This makes it harder to find smaller patterns within the 2 steps highlighted above. For this reason, a second plotting might be useful, in order

to better underline the behavior of the dataset. In order to do this, we plotted the same dataset another time while logarithmically scaling the y axis. The result of this plotting is seen in Figure 2.

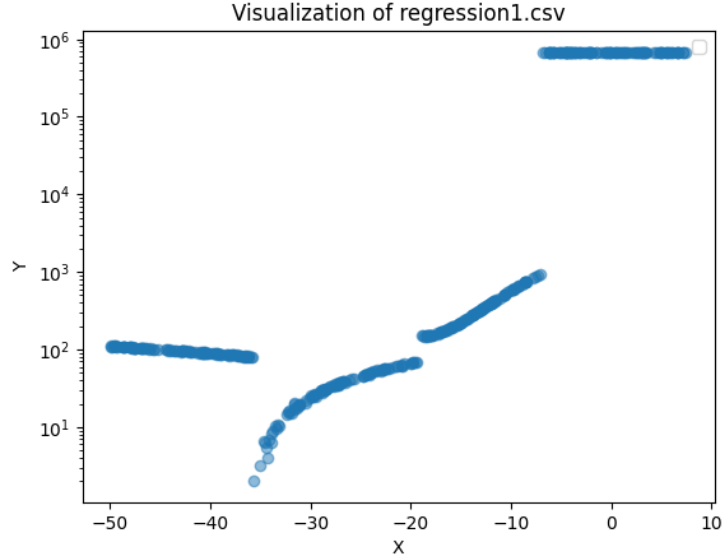


Figure 2: Visualization of regression1.csv (y-scaled)

As we can see from this new plotting, the dataset splits data into more than two different trends noticeable in the first plotting. Also notice that the second step (in the range -37 to -20) acts as a logarithmic function, which means that when it's de-scaled it acts as a rect.

Given the heterogeneous characteristics of the dataset, we'll use **Decision Trees** to correctly split the dataset into the different steps. Notice that the application of Standard Decision Tree implements them by flattening all the data within certain ranges into a constant value. This may fail to consider the linear behavior of certain steps, as seen in Figure 3.

This model doesn't get the full picture of the actual dataset. For this reason, we can try to apply a **Linear Model Decision Tree** instead of the **Standard Decision Tree**. The **Linear Model Decision Tree** is a hybrid algorithm that combines piece-wise prediction and linear modelling, providing a compound solution to our problem.

LMDT aren't included in the `sci-kit` library, so we implemented it by dividing the dataset into segments (using Decision Trees) and then applying the linear regression to the singular segments. To prevent overfitting, the Decision Tree was implemented with a `max_depth` attribute of 5. This prevents the model to consider a single data point as a segment. The results are shown in Figure 4

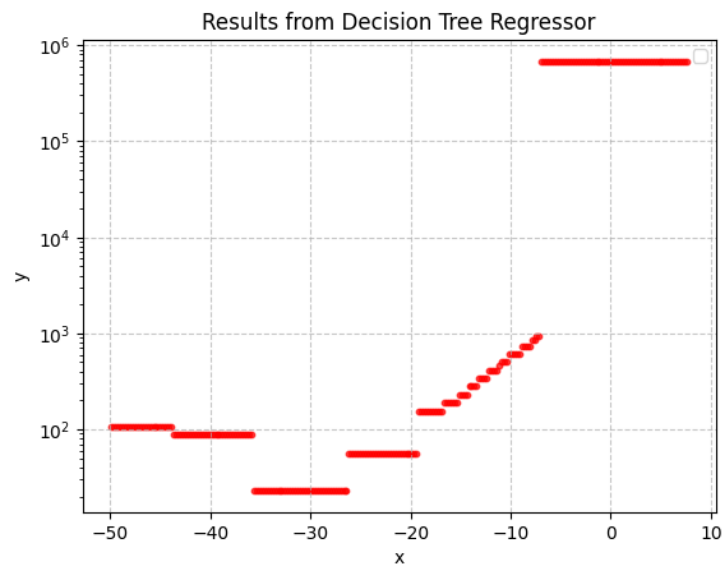


Figure 3: Visualization of the prediction model obtained through Decision Trees.

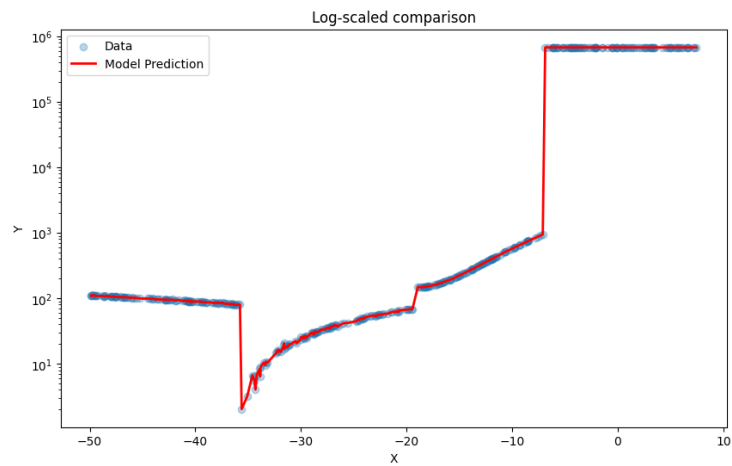


Figure 4: Comparison between raw data and model obtained through LMDT (log scaled).

2 Exercise 2

In the file `classification1.csv`, a data set with two features is collected, containing the values of elements belonging to 3 different classes. Identify the best approach to use to build a classification model for this dataset, and implement a Python script `exercise2.py` that can be used to generate a csv file (named `result2.csv`) containing the classification of the values in the square $[-45,45] \times [-45,45]$ with a step of 0.25 (the structure of the file must be similar to that of `classification1.csv`). Moreover, study the probability that a point (x,y) is associated with one of the considered classes.

As before, we plot the data to better understand class distribution. The plotting is done on a 2D space by giving every class a specific distinguishable color. The result is shown in Figure 5. As we can notice from the plotting most data are distributed in some clusters. Since they are not split in a linear-way (there are not linearly defined bounds), the application of a non-linear model might deliver the best results.

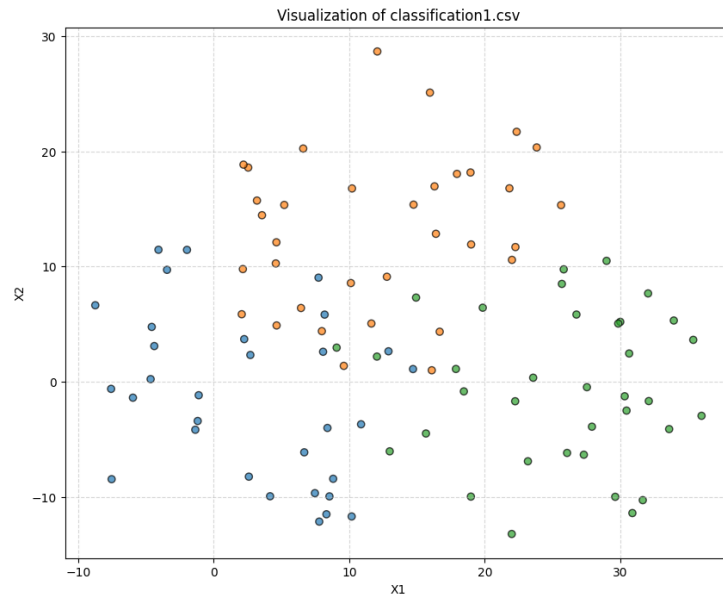


Figure 5: Visualization of `classification1.csv` enlightening the different classes

Since the classes seem to follow a circular pattern, we will not apply **Random Forest Classification**, which heavily relies on the axis defined, but we will apply **k-Nearest Neighbors** instead, which works due to the characteristics of the dataset, specifically, the closeness between same class data points. The results of this application is specific class boundaries that can be seen in Figure 6.

We also evaluate the probability distribution for all the classes using the

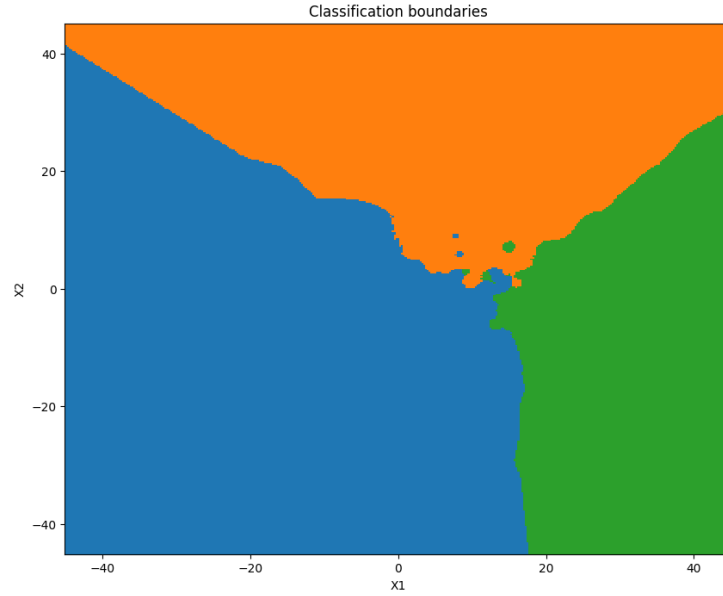


Figure 6: Visualization of results2.csv

`predict_proba` method. All the distributions are presented in Figure 7, while the combined distribution is shown in Figure 8.

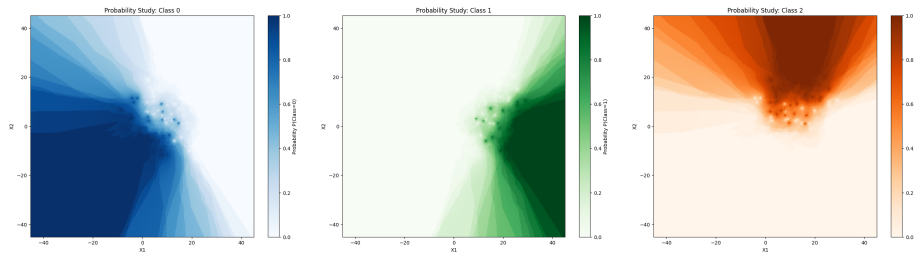


Figure 7: Probability distribution of the three classes.

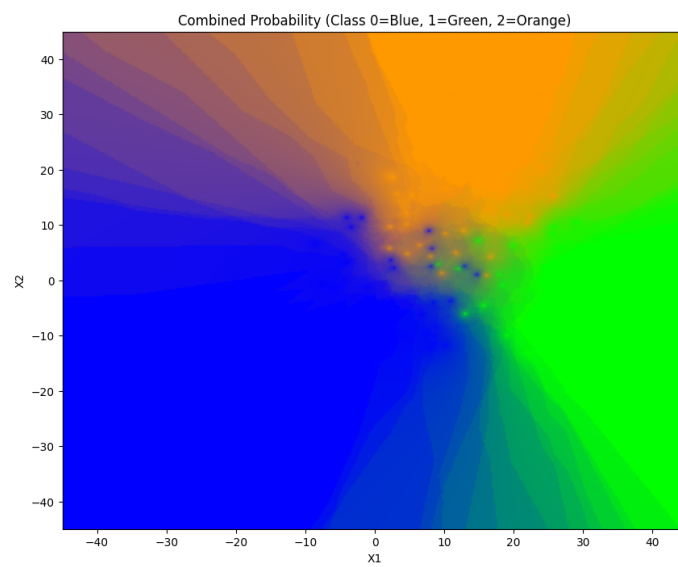


Figure 8: Combined distribution.

3 Exercise 3

In the file `regression2.csv`, a data set is collected containing the values sampled from an unknown bivariate function. Identify the best approach to use to build a regression model for this dataset, and implement a Python script `exercise3.py` that can be used to generate a csv file (named `result3.csv`) containing the values computed by the function in the $[-45,45] \times [-45,45]$ with a step of 0.25 (the structure of the file must be similar to that of `regression2.csv`).

We started by analyzing the dataset as always. One interesting aspect that emerged as part of this analysis is that there are a total of 10 values within the codomain, making the model a piece-wise constant function. Given this notion, we can consider the problem a classification problem within 10 different classes (Class 0 to Class 9). This is also shown in the 3D plotting of the dataset (where every different target value is marked in a specific color) in Figure 9 and the histogram of the frequencies in Figure 10.

3D plot

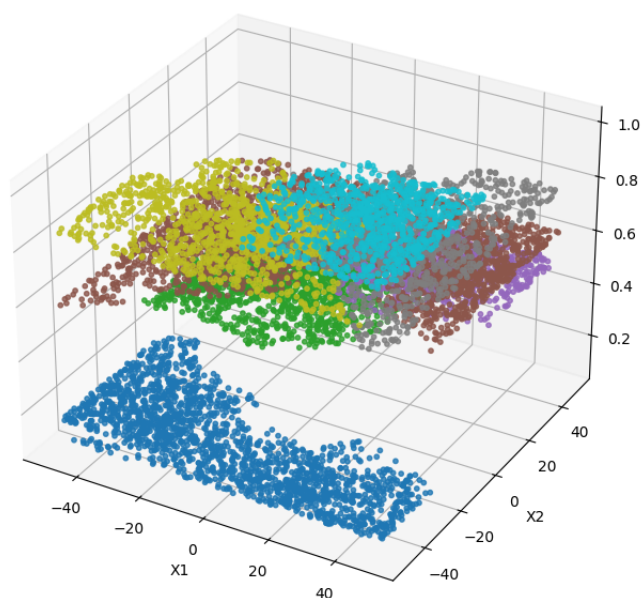


Figure 9: 3D plotting of `regression2.csv`

Since the data doesn't form a smooth surface, but it's rather divided into flat regions, we can apply a Nearest Neighbor algorithm similarly to how we did in Exercise 2. In this way, instead of applying a model to find the correct value, it just looks at the closest datapoints and acquires the same value as the target. In

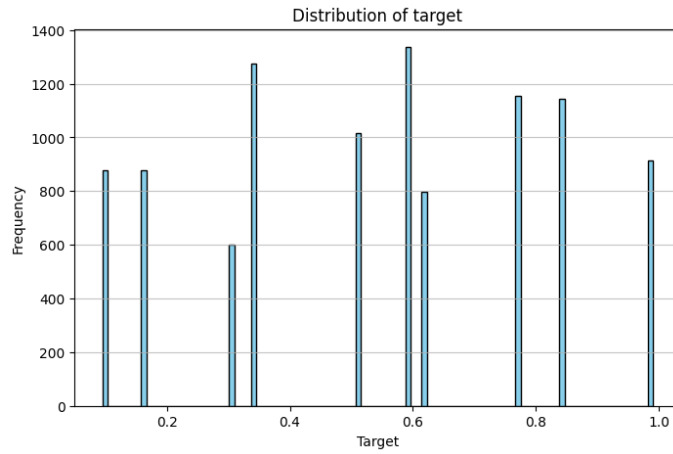
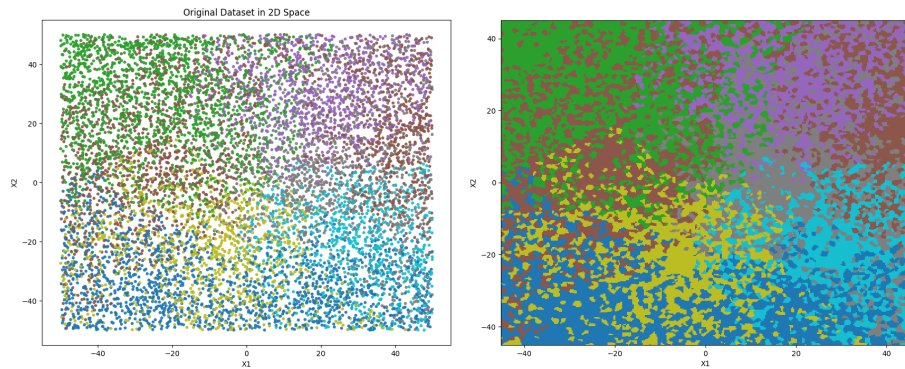


Figure 10: Histogram of frequencies

order to do this, instead of a `kNNRegressor`, that calculates the mean between the neighborhood, we can use the `kNNClassifier`, that applies the max voting algorithm, giving the datapoints the value of the k closest points. In Figure 11 are shown the plotting 2D of the original dataset and the prediction obtained after applying a 5-NN Classifier.



(a) Plotting 2D of the original dataset.

(b) Plotting 2D of the predictions.

Figure 11: Comparison between original and predicted.

4 Exercise 4

In the file `classification2.csv`, a data set with two features is collected, containing the values of elements belonging to 2 different classes. Identify the best approach to use to build a classification model for this dataset, and implement a Python script `exercise2.py` that can be used to generate a csv file (named `result2.csv`) containing the classification of the values in the square $[-45,45] \times [-45,45]$ with a step of 0.25 (the structure of the file must be similar to that of `classification1.csv`). Moreover, study the probability that a point (x,y) is associated with one of the considered classes.

Looking at the plot in Figure 12, it appears that the dataset is divided into two classes (0 - blue and 1 - red) that don't form distinct clusters, making the boundary between them more complex. The overlapping between the classes suggests that the best approach is a non-linear one. **Support Vector Machines (SVMs)** with a **Radial Basis Function (RBF)** kernel can handle the non-linearity by mapping the data into an higher-dimensional space where it is easier to properly choose a decision hyperplane to define the classes.

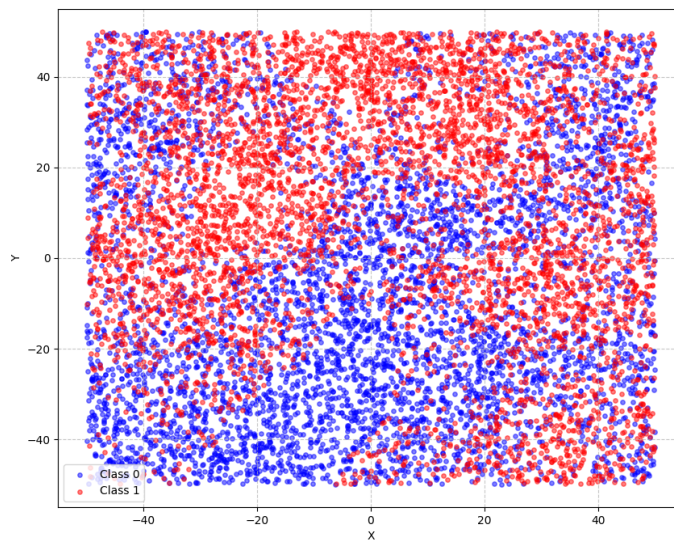


Figure 12: Plotting of the original dataset.

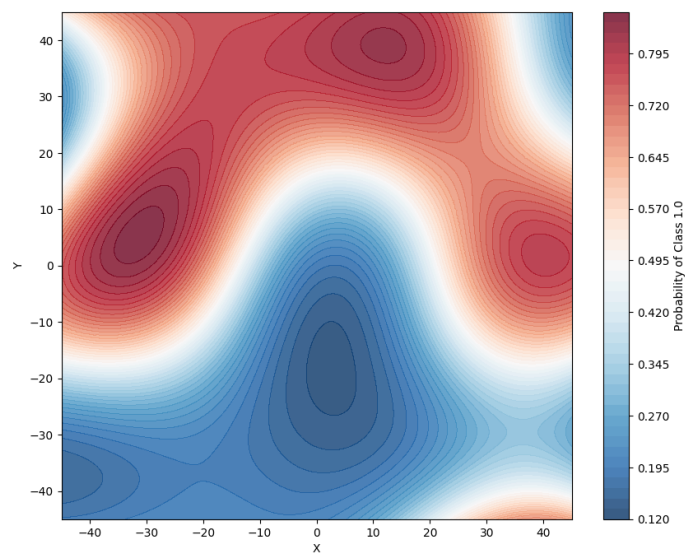


Figure 13: Probability distribution.

References

- [1] Scikit-learn. Decision trees — scikit-learn documentation. <https://scikit-learn.org/stable/modules/tree.html>.
- [2] Scikit-learn. Kneighborsclassifier — scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>.